how a `Map` could lead to object hoarding and a massive amount of uncollected garbage in some instances.

These features may not be setting your hair on fire. They're very useful for solving some specific problems, but they don't really expand your universe as a programmer. That's about to change, because it's time to explore proxies.

13.3.4. Proxies

With proxies you can give revocable access to an object. How is that useful? Imagine you're a superhero. What happens when the bad guys take you to their secret hideout? They blindfold you so that you don't know how to get there. In other words, they take you to the hideout without giving you any *reference* to it:

```
class SecretHideout
hideout = new SecretHideout()
proxy = new Proxy hideout, {}
```

Just as you can't lead the police to a hideout you can't find, there's no way you can touch a proxied object because your reference is to the proxy, not the original object:

```
proxy.policeAssault = true

hideout.policeAssault #false
```

More importantly, a proxy can capture *any* call to the object behind the proxy by defining a handler object that defines a get and set for any property accessed on the proxy. In the case of Formulaic, a form can be placed behind a proxy handler. The handler must have `get` and `set` methods that specify what happens when a property on the proxied object is accessed or modified:

```
form = document.getElementById '#the-form'
handler =
  get: -> 'property access intercepted by proxy'
  set: -> 'property modify intercepted by proxy'
proxiedForm = new Proxy form, handler
```